

OrderFlow — Project Requirements

Educational Backend Project — .NET 8

1. Project Overview

OrderFlow is a small e-commerce backend used to teach students how a real application can evolve from normal CRUD into CQRS, caching, a read-optimized Materialized View, and background processing. The project should remain small enough to implement during a course while still demonstrating real architectural trade-offs.

2. Business Scenario

Customers can browse products and create orders. The system stores transactional order data in SQL Server. Frequently requested order-summary data is exposed through a read-optimized Materialized View. The API uses caching for hot read operations. A background worker periodically refreshes the read model.

3. Functional Requirements

ID	Requirement
FR-01	The system shall create an order for a customer.
FR-02	An order shall contain one or more items.
FR-03	Each item shall contain product name, quantity, and unit price.
FR-04	The system shall calculate the order total.
FR-05	The system shall return order details by ID.
FR-06	The system shall return a list of orders for an admin/read screen.
FR-07	The system shall expose an order dashboard containing customer name, item count, total, and status.
FR-08	The system shall process pending orders in the background and move them to Completed.
FR-09	The system shall cache frequently requested order details.
FR-10	The system shall maintain a read-optimized Materialized View for the dashboard.

4. Architecture Requirements

- Use Clean Architecture with Domain, Application, Infrastructure, and API projects.
- Use Vertical Slice organization for application features.
- Use CQRS: Commands change state; Queries read data.
- Use MediatR for dispatching Commands and Queries.
- Use SQL Server as the source of truth for transactional/write data.
- Use Redis for caching.
- Implement the Materialized View as a SQL read table populated from transactional data. It is a read model, not the source of truth.
- Use BackgroundService to refresh the Materialized View periodically and process pending orders.

- Keep the implementation simple: no microservices, no Kafka/RabbitMQ, and no distributed deployment is required.

5. Suggested API

Method	Endpoint	Purpose
POST	/api/orders	Create an order (Command)
GET	/api/orders/{id}	Get order details (Query + Cache)
GET	/api/orders	List orders (Query)
GET	/api/dashboard/orders	Read dashboard from Materialized View (Query)

6. Non-Functional Requirements

- The code should be readable and suitable for classroom walkthroughs.
- Business rules must live in the domain/application flow, not in controllers.
- Read models should not be treated as the source of truth.
- Cache entries must have an expiration time.
- The system should invalidate or refresh cached order details after relevant changes.
- The background worker must create a dependency scope for scoped services.
- Configuration for SQL Server and Redis must come from appsettings/environment variables.

7. Out of Scope

Authentication/authorization, payment gateways, real message brokers, Kubernetes, event sourcing, sharding, and complex inventory management are intentionally out of scope. They can be extension exercises.

8. Expected Learning Outcomes

- Understand why CQRS separates read and write responsibilities.
- Understand the difference between a transactional database table and a read model.
- Understand why a Materialized View can make repeated dashboard queries cheaper.
- Understand what caching solves and what it does not solve.
- Understand why background work can be moved outside the HTTP request.
- Understand the trade-off between a simple solution and additional architectural complexity.